Entre Huellas: guía de proyecto, chats, sprints y GitHub

Fecha: 4 de octubre de 2026. Repositorio: https://github.com/srg-cp/entrehuellas.

Esta es la guía del MVP acordado: **Flutter + Supabase**, con MVVM, Riverpod y repositorios por funcionalidad. Adapta los sprints S1-S6 y las historias HU-01 a HU-16 del plan maestro v2. La API ASP.NET Core queda como evolución opcional.

1. Cómo dividir el trabajo entre chats

Recomendación: **un chat de coordinación, un chat de configuración y un chat por sprint**. Abrir un chat adicional por una historia grande solo cuando necesite una investigación o implementación independiente.

Chat	Propósito	Resultado
Coordinación	Este chat: alcance, arquitectura, prioridades y decisiones.	Documentación coherente y próximo incremento definido.
00 - Proyecto y estructura	Herramientas, app Flutter, carpetas, GitHub y CI.	Base ejecutable y documentada.
01 - Acceso y cuenta	S1, HU-01.	Sesión y perfil con permisos.
02 - Mascotas y zonas	S2, HU-02 a HU-04.	Mascotas, fotografías y preferencias.
03 - Perdidos y encontrados	S3, HU-05 a HU-07.	Casos en lista y mapa con filtros.
04 - QR y avistamientos	S4, HU-08 a HU-10.	Colaboración desde el navegador sin instalar.
05 - Seguimiento y recuperación	S5, HU-11 a HU-13.	Cronología, alertas y cierre.
06 - Validación del MVP	S6, HU-14 a HU-16.	Moderación, offline, métricas y demo.

Estos nombres son una propuesta de organización; la guía no crea chats automáticamente. Seleccionar el mismo proyecto/carpeta al abrir cada chat nuevo.

Mantener los archivos como registro compartido. Codex puede cargar instrucciones del proyecto desde `AGENTS.md`; las decisiones y avances que necesita el siguiente chat deben quedar escritos, sin depender de que tenga el historial anterior. Instrucciones de proyecto en Codex.

Trabajar de forma secuencial inicialmente: cerrar y validar el incremento anterior antes de comenzar uno dependiente. Si dos chats necesitan editar a la vez, usar ramas y worktrees separados; una rama diferente

en la misma carpeta no crea dos copias independientes de los archivos. Worktrees en Codex.

2. Qué construimos primero

El recorrido del MVP es:

1. El usuario inicia sesión y registra una mascota.
2. Publica un caso perdido o encontrado con fotos, hora y zona.
3. Comparte un cartel con QR.
4. Un ciudadano abre la web y reporta un avistamiento.
5. El responsable recibe ayuda en su bandeja, revisa evidencia y cierra el caso.
6. El QR muestra el estado actualizado, con moderación y manejo de conectividad.

Además: preferencias de zonas, datos públicos separados de privados, gestión de cuenta, auditoría y métricas mínimas. Chat completo, salud, cuidadores compartidos, refugios, adopciones, reservas e IA se planifican después del MVP.

Los seis sprints son una secuencia de entregas. Si se conserva la duración de dos semanas del plan, representan doce semanas referenciales. La fecha de entrega y la capacidad semanal todavía deben confirmarse; el chat 00 es preparación, no una promesa de plazo adicional fijo.

3. Estado de partida comprobado

- Git estaba inicializado. Se configuró `origin` y la rama local `main`.
- El repositorio remoto respondió sin referencias al consultarlo: vacío en esa comprobación.
- La documentación de arranque está creada. Todavía no hay app Flutter ni configuración Supabase.
- Flutter 3.35.3 y Dart 3.9.2 están instalados. No se cambió su versión.
- Para Android faltan Command-line Tools y revisar/aceptar las licencias del SDK.
- GitHub CLI no está en PATH. Se puede trabajar con Git y la interfaz web.
- No se realizaron commit, push, creación de issues, despliegues ni configuración de cuentas externas durante la preparación de esta guía.

4. Chat 00: proyecto y estructura

Orden de trabajo:

1. Leer `AGENTS.md` y los documentos; comprobar Git y conservar los archivos existentes.
2. Completar el entorno Android. En Android Studio, abrir SDK Manager y revisar SDK Tools: instalar **Android SDK Command-line Tools**, comprobar Platform-Tools, Build-Tools y una plataforma compatible. El usuario revisa los términos con `flutter doctor --android-licenses`. Volver a ejecutar `flutter doctor` y detectar un teléfono o emulador. Configuración Android oficial.
3. Elegir una versión de Flutter y dependencias compatibles. La instalada es un punto de partida; cualquier actualización se evalúa y documenta.
4. Crear Android en `apps/mobile` y organizar `lib/app`, `lib/core` y funcionalidades. Elegir el identificador Android antes de distribuir.
5. Incorporar inicialmente Riverpod, go_router y supabase_flutter. Añadir Drift/SQLite cuando se implemente el almacenamiento local. Cada paquete debe resolver una necesidad concreta.

6. Preparar configuración con ejemplos sin credenciales. Verificar la app con una pantalla inicial y estado explícito de configuración si Supabase aún no existe.
7. Crear CI con formato, análisis y pruebas pertinentes. Documentar los comandos y fijar la versión de Flutter usada por CI.
8. Revisar los archivos, realizar el primer commit y publicar la base cuando la solicitud de ejecución incluya esas acciones.

Ejemplo de comando de creación, a ejecutar en la raíz del repositorio durante este chat. `app.entrehuellas` es un prefijo provisional que debe revisarse antes de distribuir:

```
flutter create --empty --platforms=android --project-name entre_huellas --org
app.entrehuellas apps/mobile
```

Los flags fueron comprobados con el Flutter instalado. Creación de una app Flutter.

Criterio de salida: app inicial ejecutada en Android, estructura comprensible, versión registrada, comprobaciones pertinentes pasando y procedimiento para configurar Supabase documentado. Si Android sigue bloqueado, registrar lo que se verificó por otra vía y mantener el build Android pendiente.

Prompt listo para pegar en el chat 00

```
Configura el proyecto Entre Huellas en la carpeta local de este proyecto.
Lee AGENTS.md, docs/ARQUITECTURA.md, docs/ESTADO.md y la sección 4 de
docs/GUIA_MVP.md.

El repositorio es https://github.com/srg-cp/entrehuellas. Origin y la rama main
ya fueron configurados; compruébalos sin volver a clonar encima de la carpeta.

Crea la app Flutter Android en apps/mobile con MVVM, Riverpod, repositorios por
funcionalidad, go_router y supabase_flutter. Mantén Supabase como backend del
MVP. Prepara configuración de ejemplo sin secretos y CI para formato, análisis
y pruebas pertinentes.

Revisa Flutter y Android. Ya se detectaron Command-line Tools y licencias
pendientes. Ayúdame a resolver los pasos que requieran mi intervención mientras
avanzas con lo independiente. Registra la versión de SDK y el identificador
Android elegidos.

Implementa y verifica una pantalla inicial. Conserva la documentación
existente. Cuando las comprobaciones pertinentes pasen, realiza el primer
commit y publícalo en mi repositorio; no despliegues servicios. Si la
autenticación bloquea el push, deja el commit listo y explica el paso pendiente
sin pedir credenciales en el chat.

Actualiza docs/ESTADO.md con cambios, pruebas, bloqueos y el siguiente paso.
Indica si la ejecución/build Android fue realmente verificada.
```

5. Sprints e historias del MVP

Se conserva la numeración original para relacionar implementación, informe académico y pruebas. El chat 00 prepara la base técnica que el plan original situaba dentro de S1.

S1 - Acceso y cuenta

- **HU-01 / RF-01,34:** acceso por correo/Google, sesión, perfil y gestión de cuenta. Inicio válido crea o recupera un único perfil; otro usuario no puede editarlo. Solicitud de baja trazable y exportación básica de los datos del usuario según el alcance acordado.
- Crear o identificar el proyecto Supabase de desarrollo, configurar Auth y versionar esquema, permisos y migraciones.
- Comprobar sesión inválida, cierre de sesión y acceso cruzado con dos cuentas. Resolver los enlaces de retorno/confirmación de correo necesarios para Android.
- **Demo:** entrar, ver perfil, cerrar sesión y verificar permisos.

Prompt:

```
Implementa S1 de Entre Huellas según docs/GUIA_MVP.md: HU-01, RF-01 y RF-34.
Lee AGENTS.md y docs/ESTADO.md. Usa Supabase Auth y repositorios; versiona
esquema y políticas. Implementa cuenta y sesión con criterios verificables. Si
faltan URL/configuración del proyecto, solicita solo lo necesario y avanza con
código, migraciones y pruebas independientes. Actualiza el estado con la demo y
la validación real.
```

S2 - Mascotas, fotos y zonas

- **HU-02 / RF-02,16:** guardar zonas, radio y preferencias; admitir selección manual con GPS denegado y conservarlas tras reiniciar.
- **HU-03 / RF-03:** registrar y editar una mascota propia con los campos mínimos; acceso de otro usuario rechazado.
- **HU-04 / RF-03:** gestionar fotos, principal y miniaturas; controlar formato/tamaño, políticas de Storage y eliminación de EXIF en los bytes públicos.
- La solución de procesamiento de fotos debe comprobarse con los límites del runtime antes de cerrar la historia. Validar al cargar desde cámara o galería y con permisos denegados.
- **Demo:** dos usuarios, mascota privada al editar, fotografía pública permitida y zona manual persistente.

Prompt:

```
Implementa S2 de Entre Huellas: HU-02 a HU-04 y sus criterios en
docs/GUIA_MVP.md. Lee las instrucciones y el estado. Crea mascotas, fotos y
zonas con Supabase y permisos por propietario. Incluye alternativas cuando se
deniega GPS/cámara y comprueba los bytes públicos de las fotos. Verifica acceso
cruzado y persistencia. Actualiza docs/ESTADO.md con resultados y decisiones.
```

S3 - Casos perdidos y encontrados

- **HU-05 / RF-05:** publicar perdido con fecha y zona; restricción de un caso activo por mascota y creación idempotente.
- **HU-06 / RF-06:** publicar encontrado sin propietario conocido ni `pet_id` obligatorio.
- **HU-07 / RF-13:** lista y mapa con filtros coherentes y paginación; coordenadas públicas aproximadas y datos privados protegidos.
- Seleccionar el proveedor de mapas por requisitos y condiciones; habilitar PostGIS y consultar cercanía en servidor.
- Preparar desde este sprint claves estables de envío para que S6 pueda implementar la cola local sin cambiar el contrato.
- **Demo:** publicar ambos tipos, repetir un envío, filtrar y comprobar respuestas públicas.

Prompt:

```
Implementa S3: HU-05 a HU-07 de docs/GUIA_MVP.md. Usa repositorios, Supabase y
PostGIS. Cubre perdido/encontrado, lista/mapa, filtros y paginación. Impide dos
casos perdidos activos por mascota, deduplica reintentos en servidor y protege
coordenadas privadas. Verifica concurrencia y privacidad, documenta el
proveedor de mapas elegido y actualiza el estado.
```

S4 - Cartel, QR y avistamientos

- **HU-08 / RF-08,37:** cartel PDF legible, QR y compartir. Se puede generar en Flutter para el MVP; conservar enlace estable y permitir reintento de la generación.
- **HU-09 / RF-09:** crear `apps/public-web` cuando haga falta; web ligera que abre el QR y muestra estado sin instalar la app. Token revocado no revela identidad privada.
- **HU-10 / RF-11:** avistamiento desde web/app con hora del evento y punto manual, foto opcional; rechazo de fecha futura y deduplicación del envío.
- El flujo público tiene permisos acotados y validación de abuso en servidor. Acordar hosting y URL estable antes de imprimir un QR que se usará fuera de pruebas.
- **Demo:** escanear en un segundo teléfono sin app y reportar; comprobar la persistencia y la separación de campos públicos/privados.

Prompt:

```
Implementa S4: HU-08 a HU-10 de docs/GUIA_MVP.md. Crea cartel PDF/QR y una web
pública ligera en apps/public-web. El visitante debe consultar y reportar sin
instalar Flutter. Implementa tokens revocables, punto manual, hora observada,
permisos públicos limitados, controles de abuso y reintentos idempotentes.
Prepara el resultado para revisar y documenta la configuración/hosting
pendiente antes de publicar. Verifica el recorrido con otro teléfono y
actualiza el estado.
```

S5 - Seguimiento, alertas y cierre

- **HU-11 / RF-12,14:** revisar avistamientos, guardar actor/motivo y ordenar cronología por hora real.
- **HU-12 / RF-17,18:** zonas/casos seguidos, selección de destinatarios, bandeja persistente y push. Persistir trabajo de notificación con mecanismo de reintento; integrar FCM mediante operación server-side.
- **HU-13 / RF-07:** responsable autorizado cierra en una operación atómica con historial; QR actualizado y nuevas alertas de búsqueda detenidas.
- **Demo:** dejar fallar el push y encontrar la alerta en bandeja; cerrar, reabrir QR y comprobar que un usuario ajeno no puede cerrar.

Prompt:

```
Implementa S5: HU-11 a HU-13 de docs/GUIA_MVP.md. Agrega revisión/cronología,
bandeja persistente, FCM y cierre autorizado. Guarda las alertas y su trabajo
de envío antes del push, con reintentos y deduplicación. Valida estados en
Supabase y actualiza el QR al resolver. Prueba fallo del proveedor, usuario no
autorizado y cierre concurrente. Actualiza docs/ESTADO.md.
```

S6 - Moderación, offline y validación

- **HU-14 / RF-22,23,33:** denuncia, bloqueo, ocultamiento por moderador con motivo y auditoría; vía de apelación mínima.
- **HU-15 / RF-32:** borrador y caché local con Drift/SQLite, cola de envíos y estados visibles. Cortar conexión y reiniciar no pierde el borrador; reenvío no duplica ni confirma publicación antes de respuesta del servidor.
- **HU-16 / RF-35,40:** métricas agregadas y controles administrativos mínimos, accesibles solo por roles autorizados. Definir con el usuario una interfaz mínima para esta operación.
- Probar el recorrido Android/web, permisos, reintentos, conectividad y errores principales. Ensayar recuperación de datos y archivos, preparar versión de prueba y manual de demo.
- **Demo:** recuperación completa, conexión interrumpida, moderación y métricas sin información privada.

Prompt:

```
Completa S6 del MVP: HU-14 a HU-16 según docs/GUIA_MVP.md. Implementa
moderación/bloqueo/auditoría, borradores/caché y cola SQLite, y
métricas/administración mínimas. Verifica reinicio sin conexión, reenvío sin
duplicados y permisos de usuario/visitante/moderador. Ejecuta el recorrido
Android y web, documenta restauración y prepara distribución de prueba y demo.
Corrige fallos bloqueantes y registra lo que no se verificó. Actualiza
docs/ESTADO.md sin marcar completo trabajo pendiente.
```

La seguridad, pruebas y documentación acompañan cada sprint; S6 reúne la validación final.

6. GitHub con un flujo sencillo

Usar `main` como base integrada y una rama por historia o conjunto coherente de cambios. Inicialmente no hace falta mantener una rama `develop` adicional. El primer commit puede publicar la base en el repositorio vacío; los cambios posteriores siguen ramas y pull requests.

Crear un issue por HU y milestones S1-S6. La plantilla de historia está en `.github/ISSUE_TEMPLATE/historia.md`. No hace falta un tablero adicional para empezar; se puede añadir si mejora la coordinación.

Ejemplos de ramas: `chore/project-setup`, `feat/hu-01-auth`, `feat/hu-05-lost-cases`, `fix/qr-revoked-token`. El número HU no es el número del issue.

Secuencia: issue con criterios -> rama -> implementación y comprobaciones -> commit -> push -> PR -> revisión -> integración. Para trabajo individual, revisar el diff y los resultados antes de integrar. GitHub flow.

Ejemplo después de tener un `main` publicado y cuando la carpeta esté limpia. Si hay cambios pendientes, resolver su destino antes de cambiar de rama:

```
git status
git switch main
git pull --ff-only origin main
git switch -c feat/hu-01-auth
```

Tras implementar, seleccionar los archivos del cambio con `git add <rutas>` y revisar `git diff --cached`. Luego:

```
git commit -m "feat(auth): implementar acceso y sesión"
git push -u origin feat/hu-01-auth
```

Abrir la PR en GitHub utilizando `.github/pull_request_template.md`; vincular el issue real. Integrarla cuando sus criterios y checks estén resueltos. Después actualizar `main` local antes del siguiente incremento.

El repositorio ya tiene `origin`; no repetir `git remote add` ni clonar sobre la carpeta. Antes del primer push, confirmar identidad Git y autenticación. Usar el inicio de sesión del gestor de credenciales o herramientas de GitHub; mantener tokens fuera de mensajes y archivos versionados.

Los comandos de la guía son pasos para ejecutar en los chats correspondientes; su presencia no significa que ya se ejecutaron. El primer prompt autoriza explícitamente commit/push. Las siguientes tareas deben indicar las acciones de GitHub que se esperan.

7. Qué debe registrar cada chat

Al empezar: leer estado y comprobar rama/archivos. Al terminar: actualizar `docs/ESTADO.md` con:

```
Sprint / HU:
Rama y commit, si existe:
Issues / PR, si existen:
Comportamiento completado:
Migraciones y configuración:
Pruebas ejecutadas y resultados:
Pendientes o bloqueos:
Decisiones nuevas:
Siguiente paso concreto:
```

Una historia está terminada cuando sus criterios están verificados, los permisos y errores pertinentes están cubiertos, los cambios de datos están versionados y se puede demostrar el resultado. Una interfaz con datos simulados se identifica como tal; no equivale al flujo integrado.

Para trabajos de implementación con Flutter, ejecutar según corresponda desde `apps/mobile`:

```
dart format --output=none --set-exit-if-changed lib test
flutter analyze
flutter test
```

Usar `test` cuando exista; crear pruebas para comportamientos relevantes. El build Android y las pruebas en teléfono/emulador se realizan al cerrar incrementos que los requieran. Las comprobaciones del cliente no sustituyen pruebas de RLS, restricciones y RPC con usuarios distintos.

Las migraciones Supabase se guardan en el repositorio. Elegir en S1 el flujo de CLI/entornos; el stack local completo necesita sus dependencias propias y no es condición para redactar las primeras migraciones. Migraciones oficiales.

8. Prompt para una historia independiente

```
Trabaja en la historia HU-XX del sprint Sx de Entre Huellas.
Lee AGENTS.md, docs/ESTADO.md, docs/ARQUITECTURA.md y los criterios de
docs/GUIA_MVP.md.
Issue de GitHub: [enlace si existe]. Rama: [rama, si ya está creada].
Implementa el comportamiento completo con Supabase y validación pertinente.
Conserva el trabajo existente y registra pruebas y pendientes.
Actualiza docs/ESTADO.md para que el siguiente chat pueda continuar.
Acciones GitHub autorizadas en esta tarea: [indicar commit, push, issue o PR
que se espera].
```

Si la tarea implica varios chats activos, acordar quién integra y usar worktrees. Para el arranque, mantener una sola implementación activa reduce el trabajo de integración.

9. Próximo paso inmediato

Abrir **00 - Proyecto y estructura** en este proyecto y pegar su prompt. Su objetivo es dejar una base Android ejecutable y conectada a GitHub, antes de iniciar HU-01.